1. Operating System Fundamentals & Types

Question 1:
Compare and contrast multitasking and multiprogramming, considering their respective strengths and weaknesses.

Solution:
Multiprogramming and multitasking are both techniques to manage multiple processes, but they operate at different conceptual levels.

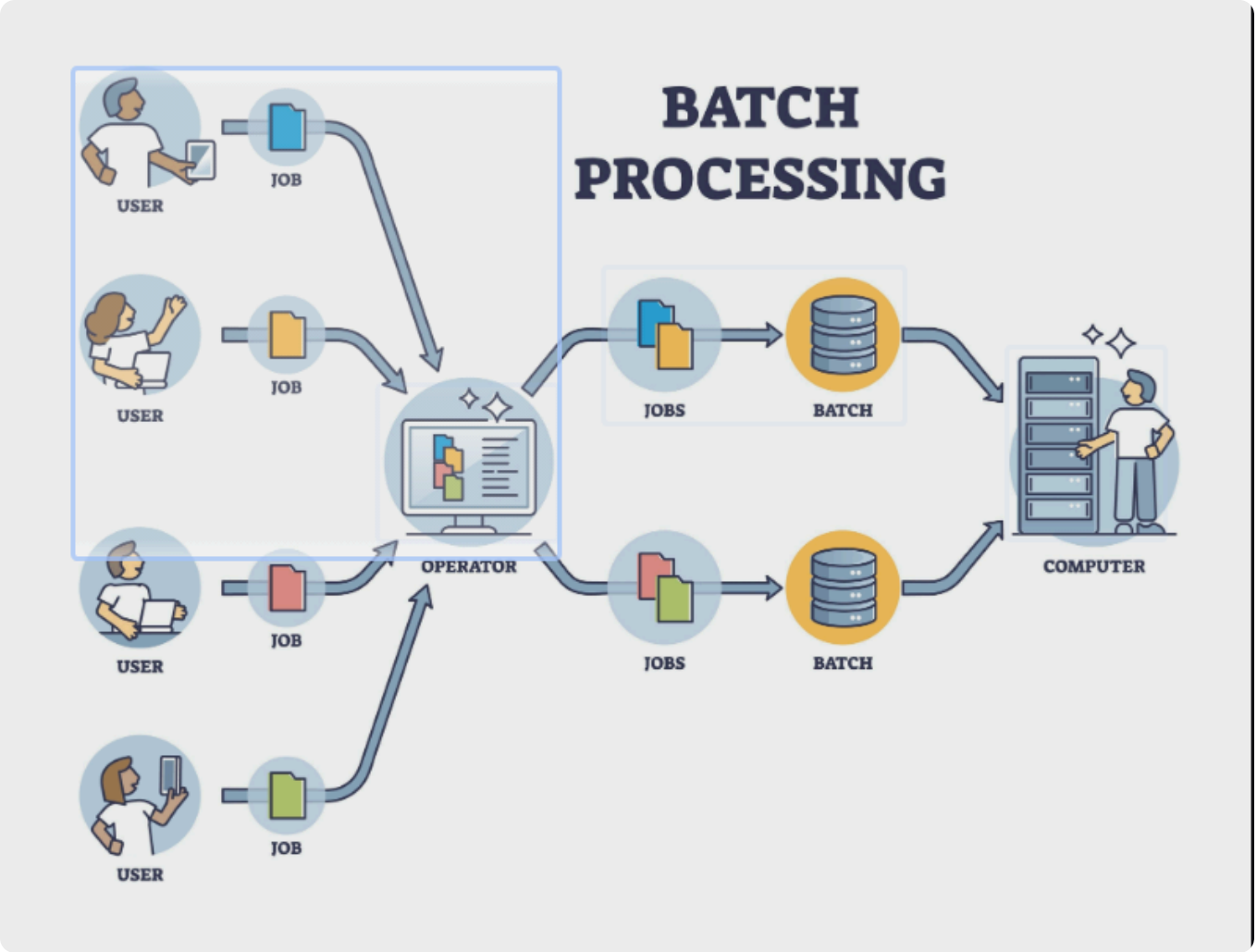
Feature	Multiprogramming	Multitasking (Time-Sharing)
Primary Goal	Maximize CPU utilization by keeping the CPU busy as much as possible.	Provide a reasonable response time to multiple users/processes simultaneously.
How it Works	CPU switches between jobs only when the current job performs an I/O operation and becomes blocked.	CPU switches between jobs based on a time quantum (time-slice), giving the illusion of simultaneous execution.
Context Switching	Triggered by I/O waits.	Triggered by the expiration of a time slice or I/O waits.
User Interaction	Low to None (often used in batch systems).	High (designed for interactive user environments).
Strength	High CPU throughput.	Excellent interactivity and fair resource sharing.
Weakness	Poor response time for interactive tasks; still requires significant user-mode interaction for each job.	Overhead from frequent context switching; can have lower raw CPU efficiency compared to batch.

Question 2:
Explain the concept of a batch operating system. Discuss its advantages and limitations in modern computing environments.

Solution:
A Batch Operating System is one where jobs (programs) are collected into batches and

executed in sequence without direct interactive control by the user during execution. The operator groups jobs with similar requirements.

- **Concept:**
  - The user prepares a job (program, data, and control information) offline (e.g., on magnetic tape or cards).
  - The OS executes jobs from a batch queue, typically on a **First-Come, First-Served (FCFS)** basis.
  - There is no direct user interaction with the job once it starts executing.



- **Advantages:**
  - **High Efficiency/Throughput:** Ideal for large, repetitive tasks that require little or no user input.
  - **Low CPU Idle Time:** The system can automatically move to the next job without waiting for user input.
  - **Resource Utilization:** Shared resources can be efficiently managed across the entire batch.
- **Limitations in Modern Computing:**

- **Lack of Interactivity:** Completely unsuitable for interactive or real-time applications.
- **Debugging Difficulty:** Errors can only be identified after the entire batch is processed.
- **High Turnaround Time:** Users may wait a long time for the output if the queue is long.
- **CPU Waste:** If a job fails early, the resources allocated to it until its completion are wasted.

Question 3:
Describe the **architecture of a multiprogramming operating system**. How does it improve CPU utilization compared to a single-program system?

Solution:

Architecture of a Multiprogramming OS

The architecture of a multiprogramming OS involves keeping **multiple jobs (processes)** concurrently in **main memory** (RAM).

1. **Job Pool:** A queue of processes ready to be loaded from disk into memory.
2. **Memory Management:** The OS must use a robust memory management scheme (like paging or segmentation) to ensure each process's memory space is protected from others.
3. **CPU Scheduler (Short-Term Scheduler):** Selects which process in the ready queue gets the CPU next.
4. **I/O Handling:** When a running process initiates an I/O operation, it cannot proceed until the I/O is complete (it becomes **blocked**).
5. **Context Switching:** When a process blocks, the OS performs a **context switch**: saving the state of the blocked process and loading the state of the next process selected by the scheduler.

Improvement in CPU Utilization

In a **single-program system**, the CPU is idle whenever the single program is waiting for an I/O operation (e.g., reading from a disk or user input). CPU utilization is low.

In a **multiprogramming system**, when one process is blocked for I/O, the OS switches the CPU to another process that is ready to run. This overlaps the **CPU execution** of one job with the **I/O wait** of another.

- **Result:** The CPU is kept busy for a much higher percentage of the time, dramatically increasing **CPU utilization** and overall system throughput.

Question 4:
Discuss **real-time operating systems (RTOS)**. Provide examples of applications where RTOS is essential.

Solution:
A **Real-Time Operating System (RTOS)** is an operating system intended to serve **real-time applications**—workloads that must be processed within a very strict, defined period. The primary measure of performance is not high throughput, but **correctness based on time constraints**.

- **Key Characteristics:**
  - **Determinism:** Operations must execute in a predictable amount of time.
  - **Low Jitter/Latency:** Minimal variation and delay in response time.
  - **Priority-based Preemptive Scheduling:** High-priority tasks immediately preempt lower-priority tasks.
  - **Fast Context Switching:** To quickly respond to time-critical events.
- **Applications where RTOS is Essential:**
  - **Industrial Control Systems:** Controlling robotic arms, factory assembly lines, and chemical processes where delays can cause physical harm or financial loss.
  - **Automotive Systems:** Engine control units (ECUs), anti-lock braking systems (ABS), and airbag deployment systems. **(Hard Real-Time)**
  - **Medical Imaging/Life Support:** MRI/CT scanners and patient monitoring systems, where timely data acquisition is crucial.
  - **Aerospace/Defense:** Flight control systems, navigation, and weapon guidance systems. **(Hard Real-Time)**
  - **Telecommunications:** Network switching equipment that must process packets within nanoseconds.

Question 5:
Describe the architecture of a **multiprocessor operating system**. How does it enhance system performance compared to a single-processor system?

Solution:
A **Multiprocessor Operating System** is designed to manage computer hardware with **multiple CPUs** (processors) sharing common resources, such as memory and peripherals.

Architecture

The two main architectures are:

1. **Symmetric Multiprocessing (SMP):**
   - All processors are **identical** and run the same copy of the operating system.
   - They share the same memory and I/O devices.
   - The OS kernel manages scheduling, I/O, and resources for **all** processors.
   - Most common type today (e.g., dual-core, quad-core CPUs).
2. **Asymmetric Multiprocessing (ASMP):**
   - Processors have specific roles (**master-slave relationship**).
   - One CPU (**master**) runs the OS kernel and handles system data structures.
   - The other CPUs (**slaves**) only execute user code as instructed by the master.

Enhancement in System Performance

1. **Increased Throughput:** By having multiple processing units, the system can execute **multiple threads or processes simultaneously**. If N processors are available, theoretically, the execution rate can be up to N times faster.
2. **Improved Reliability/Fault Tolerance:** If one processor fails, the system can often **gracefully degrade** and continue running on the remaining processors (especially in SMP).
3. **Load Sharing:** Workload can be distributed across the available CPUs, preventing bottlenecks and providing faster service to users. This is crucial for server environments and computationally intensive tasks.

Question 6:
Explain the concept of a **real-time operating system (RTOS)**. Discuss the differences between **hard real-time** and **soft real-time** systems with examples.

Solution:
A **Real-Time Operating System (RTOS)** is designed to process and respond to external events or data within strict, guaranteed time constraints. The correctness of an RTOS computation depends not only on the logical result but also on the time at which the result is produced.

Feature	Hard Real-Time System	Soft Real-Time System
Deadlines	**Absolute and critical.** Missing a deadline leads to catastrophic failure or system failure.	**Desirable, but not critical.** Missing a deadline causes performance degradation but the system can usually continue.
Timing Guarantee	**Deterministic and guaranteed.** All delays and timings are precisely bounded.	**Probabilistic.** Prioritizes critical tasks, but non-critical tasks might occasionally miss deadlines.
Latency	Extremely low and predictable interrupt latency.	Tolerates slightly higher and less predictable latency.
Examples	**Airbag deployment** in a car, **missile guidance systems**, **nuclear reactor control**, and **medical life support systems**.	**Multimedia streaming**, **online gaming**, **video conferencing**, and modern **robot vision systems**.

Question 7:
Give the **type of OS** that will be used in each case and give a reason:

- Air Traffic Control Systems
- Payroll system
- Medical imaging systems
- Server
- Robots
- Bank Statements
- Cloud

- Weapon and missile systems

Solution:

Application	Type of OS	Reason
Air Traffic Control Systems	Hard Real-Time OS (RTOS)	Critical safety demands guaranteed, immediate response to radar data and pilot inputs to prevent collisions. Failure to meet a deadline is catastrophic.
Payroll system	Batch OS (or Time-Sharing)	Runs large, repetitive calculations on scheduled intervals (e.g., end of the month). No immediate user interaction is needed during the run.
Medical imaging systems (e.g., MRI/CT)	Hard/Soft RTOS (Mixed)	Requires real-time data acquisition and processing for image reconstruction (Hard RT); viewing and manipulating the images might be Soft RT or Time-Sharing.
Server (Web/Database)	Time-Sharing / Multiuser OS	Must handle multiple client requests (processes) concurrently and provide a reasonable response time to hundreds of users simultaneously.
Robots (Industrial)	Hard Real-Time OS (RTOS)	Requires deterministic, precise timing to control motor movements and interact with the physical environment safely and accurately.
Bank Statements	Batch OS	Generating statements is a large, periodic, non-interactive process run typically overnight when resources are less utilized.
Cloud (Hypervisors/VPCs)	Distributed / Network OS	Manages a large collection of interconnected computing nodes (servers), distributing resources and providing services across a network.
Weapon and missile systems	Hard Real-Time OS (RTOS)	Failure to meet timing deadlines (e.g., in tracking, targeting, or detonation) leads to mission failure or safety hazard. Requires absolute determinism.

Kernel And System Calls

Question 1:
Discuss the role of a kernel in an operating system. Explain the differences between a monolithic kernel and a microkernel.

Solution:
The kernel is the core component of an Operating System. It is the program that has complete control over everything in the system and is responsible for managing the system's resources (CPU, memory, I/O devices) and facilitating communication between hardware and software.

Its primary roles include:

- **Process Management:** Creating, scheduling, terminating processes, and handling inter-process communication.
- **Memory Management:** Allocating and deallocating memory space for programs and managing virtual memory.
- **Device Management:** Managing I/O devices and their interaction with processes via device drivers.
- **System Calls:** Providing the interface for user processes to request OS services.
- **Interrupt Handling:** Responding to hardware and software interrupts.

Kernel Architectures

Feature	Monolithic Kernel	Microkernel
Structure	All OS services (File System, I/O, Networking, etc.) run in a single, large address space in kernel mode.	Only the most essential services (Process, Memory, and basic Inter-Process Communication (IPC)) run in kernel mode. Other services run in user space as servers.
Performance	Generally faster due to direct function calls and less overhead for communication.	Generally slower due to communication overhead (context switching and message passing via IPC) between user-space services.

Feature	Monolithic Kernel	Microkernel
Stability/Reliability	**Less reliable**; a bug in one service (like a device driver) can crash the entire kernel.	**More reliable**; a crash in a user-space service (like the file system) does not crash the entire kernel.
Extensibility	**Harder** to extend or modify; requires recompiling and rebooting the entire kernel.	**Easier** to extend; new services can be added as user-space servers without modifying the kernel.

Question 2:
Describe the process of making a **system call** in an operating system. Include the steps involved and the role of the kernel in handling the call with example.

Solution:
A **system call** is the mechanism by which a user-mode program requests a service from the operating system's kernel.

The Process of Making a System Call:

1. **User Program Invocation:** The user program executes a high-level library function (e.g., $\text{printf}()$ in C) which prepares arguments.
2. **Argument Passing:** The parameters are placed into **registers** or pushed onto the **stack**.
3. **Trap/Interrupt:** The program executes a special instruction (a **trap** or software interrupt). This switches the CPU from **user mode** to **kernel mode** and transfers control to the **System Call Handler**.
4. **Kernel Handling:** The kernel's system call handler examines the **system call number**, validates the arguments, executes the corresponding kernel-level function (e.g., file access, process creation), and performs the requested service.
5. **Return Control:** The kernel switches the CPU back to **user mode**, restores the context, and returns the result (e.g., success code or data) to the user program.

Example:
When a program calls $\text{read}()$ to read data, the kernel verifies file permissions and instructs the relevant device driver to transfer data from the device into the specified memory buffer.

Question 3:
Explain the need of various types of system call in an operating system.

Solution:
System calls are necessary because they provide a controlled, standardized, and secure interface for user programs to access and manage the protected resources of the system. Without them, user programs would have to directly manipulate hardware, which would be complex, error-prone, and highly insecure.

The various types of system calls are needed to cover the full range of OS services required by applications:

Type of System Call	Need/Purpose	Example System Calls
Process Control	To create, manage, and terminate processes, and control their execution flow.	fork(), exec(), exit(), wait()
File Management	To create, open, read, write, delete, and control file attributes.	open(), read(), write(), close(), stat()
Device Management	To request, release, read from, and write to various hardware devices.	ioctl() (I/O control), read(), write() (for device files)
Information Maintenance	To get or set system information like time, date, process ID, or OS version.	time(), getpid(), sleep()
Communication	To enable processes to exchange information (Inter-Process Communication - IPC) or establish network connections.	pipe(), shmget() (shared memory), socket(), connect()

These different types ensure that applications can perform any necessary task while maintaining the system's integrity and security under the kernel's supervision.

Question 4:
How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security) system.

Solution:
The distinction between kernel mode (privileged) and user mode (non-privileged) is a

fundamental hardware-supported protection mechanism in an Operating System.

Protection Mechanism

1. **Privilege Level:**
   - **Kernel Mode:** The CPU runs in a privileged state. All instructions, including **privileged instructions** (like I/O, memory management), can be executed.
   - **User Mode:** The CPU runs in a non-privileged state. **Only non-privileged instructions** can be executed. Any attempt by a user program to execute a privileged instruction results in a **trap** to the kernel.
2. **Resource Access:**
   - **User Mode** programs have restricted access, typically only to their own address space. They cannot directly access or modify crucial system resources (e.g., the kernel's memory space or hardware registers).
   - **Kernel Mode** has unrestricted access to *all* memory, hardware, and instructions.

Security Function

This mode distinction ensures **protection and stability**:

- **System Integrity:** It prevents a faulty or malicious user application from directly corrupting the operating system's kernel or the data of other programs.
- **Controlled Access:** It forces user programs to use the secure, verified **System Call** interface to request OS services. The kernel validates the request (e.g., checking permissions) before executing it in kernel mode.
- **Isolation:** It isolates processes from each other, ensuring a crash in one user program cannot bring down the entire system.

This mechanism strictly enforces the boundaries between application code and the core system software.

Question 5:
Explain how the $\mathrm{fork}()$ and $\mathrm{exec}()$ system calls are used together in Linux to create and execute

new processes. Then, write a simple C program that:

- Creates a child process using $\mathrm{fork}()$.
- The child process replaces itself using $\mathrm{exec}()$ to run the ls command.
- The parent process waits for the child to finish before exiting.
- Now consider the following code snippet using the $\mathrm{fork}()$ and $\mathrm{wait}()$ system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```
int x = 4;
while (x > 0) {
    fork();
    printf("hello");
    wait(NULL);
    x--;
}
```

The total number of times the printf statement is executed is

Solution:

$\mathrm{fork}()$ and $\mathrm{exec}()$ Usage

1. $\mathrm{fork}()$ **(Creation):** Creates a **child process** that is an almost exact duplicate of the calling **parent process**. The child starts execution immediately after the $\mathrm{fork}()$ call.
2. $\mathrm{exec}()$ **(Replacement):** The $\mathrm{exec}()$ family of calls **loads a new program** into the current process's address space and starts its execution. It **replaces** the child process's image with the new program (e.g., ls).

Together, they allow a process to spawn a new program for execution.

C Program Example

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
```

```c
    pid_t pid;

    // 1. Create a child process
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        // *** Child Process ***
        printf("Child process is running the ls command...\n");

        // 2. Child replaces itself with the ls program
        execlp("/bin/ls", "ls", "-l", NULL);

        // This code only runs if exec failed
        perror("exec failed");
        exit(1);
    } else {
        // *** Parent Process ***
        printf("Parent process is waiting for child to finish...\n");

        // 3. Parent waits for the child to finish
        wait(NULL);
        printf("Parent process detected child finished. Exiting.\n");
    }

    return 0;
}
```

Code Snippet Analysis

The number of processes doubles in each iteration after the $\text{fork}()$ call. Since the $\text{printf}()$ statement is executed by *every* process *after* the $\text{fork}()$, the number of prints doubles in each loop.

Iteration	Processes at Start	Processes After $\text{fork}()$	`printf` executions in this step
x = 4	1	2	2
x = 3	2	4	4
x = 2	4	8	8
x = 1	8	16	16

The total number of times the printf statement is executed is:

$$\text{Total prints} = 2 + 4 + 8 + 16 = \mathbf{30}$$

Question 6:
Describe the working of a kernel in an operating system. Explain how it manages communication between hardware and software components.

Solution:
The kernel functions as the master control program of the operating system, orchestrating all system operations. Its working can be described through its management functions:

1. **Bootstrapping and Initialization:** The kernel is the first program loaded after the BIOS/UEFI initializes the hardware. It initializes system data structures, starts crucial processes (like the scheduler and memory manager), and prepares the system for user applications.
2. **Resource Management:** It manages the four primary resources: the CPU (via the scheduler), Memory (via the memory manager), I/O devices (via device drivers), and Files (via the file system).
3. **Interrupt and Trap Handling:** It is the central recipient for all hardware interrupts (signals from devices requiring attention) and software traps (signals from processes, including system calls). The kernel suspends the current task, executes the appropriate handler routine, and then restores the state.

Hardware-Software Communication

The kernel manages communication between hardware and software through the following mechanisms:

- **System Calls:** User-level software cannot directly access hardware. It must use a system call to request a service (e.g., reading a file) from the kernel. The kernel validates the request and executes it on the software's behalf in a controlled, privileged manner.
- **Device Drivers:** These are kernel modules that contain the hardware-specific code required to communicate with a particular device (e.g., a printer or network card). When a process requests I/O, the kernel routes the request to the relevant device driver, which translates the request into commands the hardware understands.

- **Interrupts:** Hardware components signal the kernel using interrupts when they need attention (e.g., a network card has received a packet, or a disk drive has finished an operation). This allows the hardware to communicate *asynchronously* with the kernel, preventing the CPU from constantly polling devices.

Question 7:
Differentiate between a **System Call** and a **Library Function** (like a C standard library function). Provide a concrete example of a library function that does *not* result in a system call.

Solution:
The difference lies primarily in the security domain, privilege level, and interaction with the operating system kernel.

Feature	System Call	Library Function (Standard C Library)
Purpose	Provides the mandatory interface for user programs to access **protected OS resources** (e.g., files, network, memory).	Provides a convenient, high-level, and portable way to implement common functions and data structures.
Execution Mode	Executes in **Kernel Mode** (privileged). Involves a **context switch** from User Mode to Kernel Mode.	Executes entirely in **User Mode** (unprivileged).
Portability	Generally **less portable** as they are specific to the underlying OS (e.g., Linux $\text{fork}()$ vs. Windows $\text{CreateProcess}()$).	Generally **highly portable** across different OS platforms.
Mechanism	Invokes a **software interrupt** or **trap** instruction to transfer control to the kernel.	Simple function call (JUMP/CALL) within the process's own address space.
Dependency	May be called directly by a library function.	May or may not use a system call, depending on its purpose.
Examples	$\text{read}()$, $\text{write}()$, $\text{open}()$, $\text{mmap}()$.	$\text{printf}()$, $\text{scanf}()$, $\text{strcat}()$, $\text{sqrt}()$.

Library Function without a System Call

A concrete example of a standard C library function that generally does **not** result in a system call is:

- `strcat()` (String concatenation)
- `abs()` (Absolute value calculation)
- `toupper()` (Convert character to uppercase)

These functions only manipulate data or perform computations *within* the process's existing memory space and do not require interaction with the kernel, I/O devices, or other protected resources.

Question 8:
Describe the concept of **dual-mode operation** and why it is essential for the protection and security of an operating system.

Solution:

Dual-Mode Operation

Dual-mode operation refers to the hardware mechanism in which the CPU can operate in one of two distinct states or modes:

1. **User Mode (Least Privileged):** The CPU executes code on behalf of the user application. Certain instructions, designated as **privileged instructions** (like I/O control, timer management, or modifying memory access protection), are strictly disallowed.
2. **Kernel Mode (Most Privileged / System Mode):** The CPU executes code on behalf of the Operating System kernel. All instructions, including privileged instructions, can be executed.

The CPU relies on a **Mode Bit** (a single bit in a hardware register) to indicate the current operating mode. A hardware interrupt, trap, or system call causes the mode bit to switch from User to Kernel. A special instruction is used by the kernel to switch the bit back from Kernel to User before returning control to the user program.

Essential for Protection and Security

Dual-mode operation is essential because it is the **hardware-level foundation** for OS protection:

- **Prevents Accidents and Malice:** It prevents a faulty or malicious user program from directly manipulating critical system resources. For example, a user program cannot arbitrarily halt the machine, format the disk, or modify the kernel's code because those instructions are privileged and trapped when executed in User Mode.
- **Controlled Access:** It forces every request for a critical OS service (like file access or memory allocation) to go through the **System Call** interface. This trap mechanism gives the kernel the opportunity to **authenticate** the request and check the user's permissions *before* running the privileged code.
- **System Integrity:** It ensures that the kernel's own memory and data structures are protected from being overwritten by unprivileged user code, thereby maintaining the **stability and integrity** of the entire system.

Question 9:
What is a **Trap** in the context of system calls, and how does it facilitate the transition of control from user code to the kernel?

Solution:
A **trap** (or software interrupt) is a specific type of software-generated interrupt used by a user process to request a service from the operating system kernel.

Role and Function

1. **Transfer of Control:** The trap instruction is the essential mechanism that facilitates the controlled and immediate transition of execution control from **User Mode** to **Kernel Mode**.
2. **System Call Interface:** When a user program intends to execute a system call (e.g., $\text{open}()$), the library function sets up the required parameters and the system call number in registers. It then executes the trap instruction.
3. **Mode Switch:** The execution of the trap instruction has the following direct hardware-enforced effects:
   - It immediately switches the CPU's hardware **Mode Bit** from User to Kernel.
   - It saves the state of the currently running user program (registers, program counter).

- It transfers control to a fixed location in the kernel's memory space, known as the **Interrupt Vector** or **System Call Handler** entry point.

Once in the System Call Handler, the kernel examines the system call number to determine which requested service needs to be executed. This mechanism is critical because it is the *only* legal way for user code to switch into the highly privileged kernel mode.

Question 10:
List and briefly describe the five major categories of System Calls, providing a Linux/Unix example for each category.

Solution:
The five major categories of system calls are designed to cover the principal functions of an operating system:

1. **Process Control:**
   - **Description:** Used to manage the creation, termination, suspension, loading, and execution flow of processes. Includes functions for retrieving process information and managing execution context.
   - **Example:** $\mathrm{fork}()$ (creates a child process), $\mathrm{exit}()$ (terminates the calling process).
2. **File Management:**
   - **Description:** Used to manage file systems and interact with files. This includes creating, deleting, opening, closing, reading, writing, and repositioning the file pointer.
   - **Example:** $\mathrm{open}()$ (opens a file and returns a file descriptor), $\mathrm{read}()$ (reads data from a file descriptor).
3. **Device Management:**
   - **Description:** Used to request, release, and control access to physical and virtual I/O devices (e.g., disk drives, network interfaces, screens).
   - **Example:** $\mathrm{ioctl}()$ (performs device-specific I/O operations), $\mathrm{read}()$ and $\mathrm{write}()$ are often used for device files as well.
4. **Information Maintenance:**
   - **Description:** Used to transfer information between the user program and the operating system. This includes getting and setting system date/time, process identifiers (PID), and general system data.
   - **Example:** $\mathrm{getpid}()$ (returns the process ID of the calling process), $\mathrm{time}()$ (returns the current time).

5. **Communication:**
    - **Description:** Used for Inter-Process Communication (IPC) to allow processes to exchange information, either on the same machine or across a network.
    - **Example:** $\text{pipe}()$ (creates a channel for communication), $\text{socket}()$ (creates a network communication endpoint).

Linux And Shit

Linux Commands and Shell Scripting 🖥️

Question 1:
Write and explain the Linux commands to:

- Display the present working directory
- Create, view, and navigate directories
- Display system information

Solution:

- **Display the present working directory:**
    - Command: `pwd`
    - Explanation: Stands for **P**rint **W**orking **D**irectory. It shows the full path of the directory you are currently working in.
- **Create, view, and navigate directories:**
    - **Create:** `mkdir my_folder`
        - Explanation: Stands for **M**a**k**e **Dir**ectory. It creates a new folder (directory) named `my_folder`.
    - **View:** `ls`
        - Explanation: Stands for **L**i**s**t. It shows all the files and folders inside the current directory.
    - **Navigate:** `cd my_folder`
        - Explanation: Stands for **C**hange **D**irectory. It moves you into the `my_folder`. Use `cd ..` to move back up one level.
- **Display system information:**
    - Command: `uname -a`
    - Explanation: Shows general information about your operating system (like the kernel version and type). The `-a` flag means "all" details.

Question 2:
Write a shell script that uses a function to calculate the factorial of a number entered by the user, but only if the number is prime. If the entered number is not prime, display an appropriate message.

Solution:

```
#!/bin/bash

# Function 1: Check if a number is prime (simple logic)
is_prime() {
    local n=$1
    # Check for numbers less than or equal to 1
    if [ $n -le 1 ]; then
        return 1 # Not prime (exit status 1)
    fi
    # Loop from 2 up to the number-1
    for ((i=2; i<n; i++)); do
        # Check if remainder is 0 (meaning it's divisible)
        if [ $((n % i)) -eq 0 ]; then
            return 1 # Not prime
        fi
    done
    return 0 # Prime (exit status 0)
}

# Function 2: Calculate factorial
calculate_factorial() {
    local n=$1
    local fact=1
    # Loop from 1 up to the number
    for ((i=1; i<=n; i++)); do
        fact=$((fact * i))
    done
    echo $fact
}

# Main Script
echo "--- Factorial Script ---"
read -p "Enter a number: " num

# Call the prime check function
```

```
if is_prime $num; then
    # If prime, call the factorial function
    result=$(calculate_factorial $num)
    echo "$num is a prime number."
    echo "Factorial of $num is: $result"
else
    echo "$num is NOT a prime number. Factorial calculation skipped."
fi
```

Question 3:
Discuss the importance of shell scripting in Linux. Provide an example of a script that uses variables, loops, and conditional statements.

Solution:

Importance of Shell Scripting

Shell scripting is important because it allows you to automate repetitive tasks in Linux. Instead of typing the same set of commands manually every day (like backing up files or checking system health), you can write them into a script file and run them with a single command. This saves time, reduces errors, and makes complex administrative tasks much easier. It's the primary tool for system automation and management.

Example Script (Simple File Checker)

This script checks if a user's file exists and reports the status.

```
#!/bin/bash

# Variable: Store the name of the file to check
FILENAME="report.txt"

echo "Checking if the important file $FILENAME exists..."

# Conditional Statement (IF statement)
# -f checks if a regular file exists
```

```
if [ -f "$FILENAME" ]; then
    echo "SUCCESS: The file is found!"
else
    echo "ERROR: The file $FILENAME is missing."
fi

# Loop: Create dummy files for demonstration
echo "--- Creating 3 dummy log files ---"
for i in 1 2 3; do
    # Variable 'i' changes in each loop
    TOUCH_NAME="log_$i.tmp"
    touch "$TOUCH_NAME"
    echo "Created $TOUCH_NAME"
done
```

Question 4:
Complete the following shell script to find the average of numbers given at the command line.

```
#!/bin/bash
sum=0
count=$#
for num in "$@"
do
sum=$(( # Missing calculation
done
avg=$(( # Missing calculation
echo "Average = $avg"
))
```

Solution:
The completed script for finding the integer average:

```
#!/bin/bash
sum=0
count=$#

# $# holds the count of arguments (numbers) given by the user
# $@ holds all the arguments given by the user

# Safety check: Prevent division by zero if no numbers are provided
```

```
if [ $count -eq 0 ]; then
    echo "Please provide numbers as command line arguments (e.g., ./script.sh 10 20 30)"
    exit 1
fi

for num in "$@"
do
    # Missing calculation: Add the current number 'num' to the running 'sum'
    sum=$(( sum + num ))
done

# Missing calculation: Divide the total 'sum' by the 'count' of numbers
# Note: Shell scripting performs integer division (it ignores the remainder/decimal part)
avg=$(( sum / count ))

echo "Average = $avg"
```

Question 5:
Describe the role of the **Command Line Interface (CLI)** in Linux compared to the **Graphical User Interface (GUI)**. Also summarize the purpose of basic Linux commands such as `ls`, `ls -l`, `mkdir`, `rmdir`, `cp`, and `rm` in Linux.

Solution:

Role of CLI vs. GUI in Linux

- **Command Line Interface (CLI):** This is the **text-based** way to interact with the computer (the Terminal/Shell). Its role is primarily for **speed, power, and automation**. It is essential for system administrators, running scripts, and managing servers remotely because it uses very few resources.
- **Graphical User Interface (GUI):** This is the **visual** way to interact with the computer (like the desktop environment with icons, windows, and a mouse). Its role is to provide an **easy, intuitive, and beginner-friendly** environment for everyday tasks like browsing the web or viewing pictures.

Purpose of Basic Linux Commands

Command	Purpose Summary
ls	List files and folders in the current directory.
ls -l	List files in a long detailed format (showing permissions, size, etc.).
mkdir	Make Directory (used to create a new folder).
rmdir	Remove Directory (used to delete an empty folder).
cp	Copy files or folders from one location to another.
rm	Remove files or folders (used to delete them permanently).

Question 6:
Write a shell script to calculate the Fibonacci sequence. Explain each step of the script.

Solution:

Fibonacci Sequence Script

This script asks the user for the number of terms they want and generates the sequence.

```
#!/bin/bash

# 1. Read the number of terms from the user
read -p "Enter the number of Fibonacci terms to generate: " N

# 2. Initialize the first two terms of the sequence
# 'a' is the first term (0)
a=0
# 'b' is the second term (1)
b=1
# 'i' is the loop counter, starting at 2 since the first two terms are already
defined
i=2

```

```
14  echo "Fibonacci Series up to $N terms:"
15
16  # 3. Print the first term if N >= 1
17  if [ $N -ge 1 ]; then
18      echo -n "$a "
19  fi
20
21  # 4. Print the second term if N >= 2
22  if [ $N -ge 2 ]; then
23      echo -n "$b "
24  fi
25
26  # 5. Loop to calculate the rest of the terms
27  # The loop runs from the 3rd term (i=2) up to the number N
28  while [ $i -lt $N ]; do
29      # Calculate the next term (c) by adding the previous two (a + b)
30      c=$((a + b))
31
32      # Print the calculated term
33      echo -n "$c "
34
35      # Update the variables for the next iteration:
36      # 'a' becomes the old 'b'
37      a=$b
38      # 'b' becomes the new term 'c'
39      b=$c
40
41      # Increment the loop counter
42      i=$((i + 1))
43  done
44
45  echo "" # Print a final newline for clean formatting
```

Step	Script Line(s)	Explanation
Initialization	`a=0` , `b=1`	Sets the two starting numbers of the sequence (0 and 1).
User Input	`read -p "..." N`	Asks the user how many numbers they want and stores it in variable N.
Output Start	`if [ $N -ge 1 ];...`	Prints the starting numbers (0 and 1), checking if the user asked for at least 1 or 2 terms.
Loop Start	`while [ $i -lt $N ];`	Starts a loop that continues until the required number of terms (N) is reached.
Calculation	`c=$((a + b))`	Calculates the next Fibonacci number by adding the two previous numbers.

Step	Script Line(s)	Explanation
Printing	`echo -n "$c "`	Displays the newly calculated term.
Shifting	`a=$b`, `b=$c`	**Shifts the values:** The current second term (`b`) becomes the new first term (`a`), and the new term (`c`) becomes the second term (`b`) for the next cycle.
Increment	`i=$((i + 1))`	Increases the counter to move to the next term number.

Question 7:
Predict the output of the following shell script when executed as:
`./script.sh 5928`

```
#!/bin/bash
num=$1
pos=1
while [ $num -ne 0 ]
do
    digit=$((num % 10))
    if [ $((pos % 2)) -ne 0 ]
    then
        # Missing code
    fi
    echo -n "$digit "
    num=$((num/10))
    pos=$((pos+1))
done
echo
```

Solution:

Output Prediction

The script processes the number **5928** digit by digit from **right to left** (least significant to most significant).

1. The loop continues as long as `num` is not zero.
2. `digit = num % 10` extracts the last digit.
3. `num = num / 10` removes the last digit (integer division).
4. `pos` tracks the position of the digit, starting at 1 (odd position).
5. The missing code is inside a condition that checks if the position is **odd** (`pos % 2 != 0`).
6. *Since the question asks to predict the output without knowing the missing code, we must assume the missing code does not affect the final output line (`echo -n "$digit "`) or that its effect is irrelevant to the final print logic.*

The key is the `echo -n "$digit "` line, which prints the digit followed by a space, **regardless** of the `if` condition.

Iteration	num **(Start)**	pos **(Start)**	$\text{digit} = \text{num}\%10$	$\text{pos}\%2 \neq 0$ **?**	**Output Printed**	num **(End)**	pos **(End)**
1	5928	1	8	True (Odd)	`8`	592	2
2	592	2	2	False (Even)	`2`	59	3
3	59	3	9	True (Odd)	`9`	5	4
4	5	4	5	False (Even)	`5`	0	5

The final `echo` adds a newline.

Predicted Output:

```
8 2 9 5
```

Question 8:
Write a shell script to display the digits which are in **odd position** in a given 5-digit number.
Explain each step of the script.

Solution:

Odd Position Digits Script

This script uses the same logic as Question 7, but focuses the output only on the digits found in odd positions (1st, 3rd, 5th from the right).

```bash
#!/bin/bash

# --- Input and Validation ---
read -p "Enter a 5-digit number: " num

# Basic check to ensure it's exactly 5 digits
if [[ ! "$num" =~ ^[0-9]{5}$ ]]; then
    echo "Error: Please enter a valid 5-digit number."
    exit 1
fi

echo "Digits in odd positions (from right to left):"

# --- Processing Logic ---
original_num=$num # Store for display purposes
position=1        # Start counting position from 1 (the rightmost digit)

# Loop: Continue as long as there are digits left (num is not 0)
while [ $num -ne 0 ]; do

    # 1. Extract the last digit: Uses modulo 10
    digit=$((num % 10))

    # 2. Check the position: True for 1st, 3rd, 5th, etc.
    if [ $((position % 2)) -ne 0 ]; then
        # 3. Display the digit if the position is odd
        echo -n "$digit "
    fi

    # 4. Remove the last digit: Uses integer division by 10
    num=$((num / 10))

    # 5. Increment the position counter for the next digit
    position=$((position + 1))
done

echo "" # Final newline
```

Step	Script Line(s)	Explanation
Initialization	`position=1`	Starts the counter for the digit position from the right. `1` is an odd position.
Loop Start	`while [ $num -ne 0 ];`	Loop processes the number until all digits are extracted.
Digit Extraction	`digit=$((num % 10))`	Isolates the rightmost digit. Example: $54321 \rightarrow 1$.
Conditional Check	`if [ $((position % 2)) -ne 0 ];`	Checks if the current position number (1, 2, 3...) is odd.
Display Digit	`echo -n "$digit "`	Prints the digit only if its position is odd.
Number Update	`num=$((num / 10))`	Removes the rightmost digit from the number. Example: $54321 \rightarrow 5432$.
Position Update	`position=$((position + 1))`	Moves the counter to the next position (2, 3, 4...).